Final Project - Intro to Data Science

2026-09-29

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.1     v readr     2.2.0
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.3     v tibble    3.3.1
## v lubridate 1.9.5     v tidyr     1.3.2
## v purrr     1.2.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(ComplexHeatmap)
```

```
## Loading required package: grid
## ========================================
## ComplexHeatmap version 2.28.0
## Bioconductor page: http://bioconductor.org/packages/ComplexHeatmap/
## Github page: https://github.com/jokergoo/ComplexHeatmap
## Documentation: http://jokergoo.github.io/ComplexHeatmap-reference
##
## If you use it in published research, please cite either one:
## - Gu, Z. Complex Heatmap Visualization. iMeta 2022.
## - Gu, Z. Complex heatmaps reveal patterns and correlations in multidimensional
##     genomic data. Bioinformatics 2016.
##
##
## The new InteractiveComplexHeatmap package can directly export static
## complex heatmaps into an interactive Shiny app with zero effort. Have a try!
##
## This message can be suppressed by:
##   suppressPackageStartupMessages(library(ComplexHeatmap))
## ========================================
```

```
library(circlize)
```

```
## ========================================
## circlize version 0.4.18
## CRAN page: https://cran.r-project.org/package=circlize
## Github page: https://github.com/jokergoo/circlize
## Documentation: https://jokergoo.github.io/circlize_book/book/
##
## If you use it in published research, please cite:
## Gu, Z. circlize implements and enhances circular visualization
##   in R. Bioinformatics 2014.
##
```

```
## This message can be suppressed by:
##   suppressPackageStartupMessages(library(circlize))
## ========================================
```

```
library(uwot)
```

```
## Loading required package: Matrix
##
## Attaching package: 'Matrix'
##
## The following objects are masked from 'package:tidyr':
##
##     expand, pack, unpack
```

1. **Downloaded data**

```
# Explanation: I want to load the count data. I also want to generate some
# initial variables that I expect to use across code blocks.

tcga_data <- read.csv(file = "counts.csv", row.names = 1) # Read data from
#"counts" CSV

# Check for any NAs in counts
sum(is.na(tcga_data))
```

```
## [1] 0
```

```
esr1_gene <- tcga_data["ENSG00000091831",] # Instantiate expression values
# for my gene of choice

esr1_numeric <- as.numeric(esr1_gene) # Instantiate numeric vector of
# expression values for use throughout interpretations, as needed

erbb2_gene <- tcga_data["ENSG00000141736",] # Instantiate expression
#values for my other gene of choice

erbb2_numeric <- as.numeric(erbb2_gene) # " " re: expression values
```

2) **Summary Statistics**

```
# Explanation: I want to generate summary statistics in a tidy way

summary_stats <- tibble(expression = esr1_numeric) %>% # Create data frame
  # of numeric vector
  summarize( # Generate summary statistics with summarize() function
    mean = mean(expression, na.rm = TRUE),
    median = median(expression, na.rm = TRUE),
    sd = sd(expression, na.rm = TRUE),
    min = min(expression, na.rm = TRUE),
    max = max(expression, na.rm = TRUE)
  )

summary_stats # Return the statistical data
```

```
## # A tibble: 1 x 5
##     mean median     sd   min    max
##    <dbl>  <dbl>  <dbl> <dbl>  <dbl>
```

```
## 1 31442.  17561 38261.    14 270977
```

3) **Visualization**

```r
# Explanation: I want a function to create the histogram, scatter, and violin plots. Since each plot fu

library(ggpubr) # Since not part of the tidyverse, and we need ggarrange() below

create_plots <- function(first_gene_expression,
                         first_gene_name,
                         second_gene_name,
                         second_gene_numeric,
                         metadata,
                         fill_or_color = "#1B9E77") {

  # Because we want to make a new vector from the argument passed
  first_gene_numeric <- as.numeric(first_gene_expression)

  # Histogram
  hist_plot <- ggplot(
    data.frame(expression = first_gene_numeric),
    aes (x = expression)
  ) +

    geom_histogram(bins = 30,
                   alpha = 0.8,
                   fill = fill_or_color,
                   color = "black") +
    labs(
      title = paste("Distribution of", first_gene_name, "Expression"),
      x = "Gene explression value",
      y = "Frequency"
      ) +
    theme_minimal() +
    theme(plot.margin = margin(15, 15, 15, 15), # I want the plot to have
          # padding, including above title
          plot.title = element_text(face = "bold",
                                    size = 12,
                                    margin = margin(t = 20, b = 10)),
          axis.title = element_text(size = 10),
          axis.text = element_text(size = 8)) +
    scale_x_continuous(breaks = seq(0, 300000, by = 50000))


  # Scatter plot
  scatter_df <- data.frame(
    Gene_1 = first_gene_numeric,
    Gene_2 = second_gene_numeric
  )

  scatter_plot <- ggplot(
    scatter_df,
    aes(x = Gene_1, y = Gene_2)
  ) +
    geom_point( # Apply scatter geometry
```

```
  color = fill_or_color,
  alpha = 0.6
) +

  # Add a regression line to the scatter plot
  geom_smooth(
    method = "lm",
    se = FALSE,
    color = "red",
    linewidth = 1) +

  # Add correlation annotation to show significance (bottom right)
  # I learned this method from Microsoft Copilot
  stat_cor(
    label.x.npc = 0.25,
    label.y.npc = 0.10,
    size = 3,
    color = "red") +

# Since the expression ranges are so large, I want to log them in order
# to visualize/interpret the data more efficiently
scale_x_log10(
  breaks = c(10, 100, 1000, 10000, 100000),
  labels = c("10", "100", "1,000", "10,000", "100,000")
) +
scale_y_log10(
  breaks = c(1000, 10000, 100000, 1000000),
  labels = c("1,000", "10,000", "100,000", "1,000,000")
) +
labs(
  title = paste(first_gene_name, "vs", second_gene_name, "Expression"),
  x = paste(first_gene_name, "Expression (log10)"),
  y = paste(second_gene_name, "Expression\n(log10)")
) +
theme_minimal() +
theme(plot.margin = margin(5, 5, 5, 5),
      plot.title = element_text(face = "bold",
                                size = 12,
                                margin = margin(t = 20, b = 10)),
      axis.title = element_text(size = 10),
      axis.text = element_text(size = 8))


# Violin Plot
violin_data_frame <- tibble(

  # head(names(esr1_gene)) returns samples with a period separator. In
  # order to match the metadata barcodes, we have to turn each period
  # into a hyphen
  barcode = gsub("\\.", "-", names(first_gene_expression)),
  expression = first_gene_numeric
  ) %>%
  inner_join(
```

```
      metadata %>%
      select(barcode, ajcc_pathologic_stage),
      by = "barcode"
      ) %>%

    # The data have some NAs and X's in the ajcc_pathologic_stage column,
    # which we want to filter out
    filter(!is.na(ajcc_pathologic_stage),
           ajcc_pathologic_stage != "Stage X")

  violin_plot <- ggplot(violin_data_frame,
                        aes(x = ajcc_pathologic_stage,
                            y = expression)) +
    geom_violin(fill = fill_or_color,
                alpha = 0.7) +
    geom_boxplot( # Add boxplots to visualize distribution of data,
      # with outliers colored red
      width = 0.1,
      fill = "white",
      outlier.size = 0.5,
      outlier.color = "red") +
    scale_y_log10(
      breaks = c(10, 100, 1000, 10000, 100000),
      labels = c("10", "100", "1,000", "10,000", "100,000")
      ) +
    labs(title = paste(first_gene_name, "Expression by Pathologic Stage"),
         x = "Pathologic Stage",
         y = paste(first_gene_name, "Expression\n(log10)")) +
    theme_minimal() +
    theme(
      axis.text.x = element_text(
        angle = 45,
        hjust = 1,
        size = 8),
      plot.margin = margin(15, 15, 15, 15),
      plot.title = element_text(face = "bold",
                                size = 12,
                                margin = margin(t = 20, b = 10)),
      axis.title = element_text(size = 10),
      axis.text = element_text(size = 8))

  # Arrange plots
  all_plots <- ggarrange(ggarrange(hist_plot, scatter_plot,
                                   ncol = 2,
                                   labels = c("Figure 4", "Figure 5")),
                         violin_plot,
                         nrow = 2,
                         heights = c(1, 1.5), # Make violin plot larger
                         labels = c("", "Figure 6"))

  return(all_plots)
}
```

#'s 3A. through 3C.

```r
# Explanation: Now, I need to call the function with the data for each plot to
# generate the final plots. Thereafter, I save the plots to a PNG.

# I need to fetch the metadata to generate the violin plot
tcga_meta_data <- read.csv(file = "meta_data.csv")

final_plots <- create_plots(
  first_gene_expression = esr1_gene,
  first_gene_name = "ESR1",
  second_gene_name = "ERBB2",
  second_gene_numeric = erbb2_numeric,
  metadata = tcga_meta_data
)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

```r
final_plots
```

Figure 4

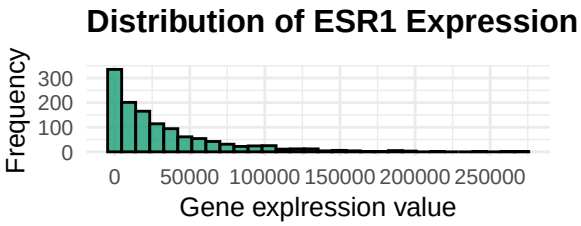


Figure 5

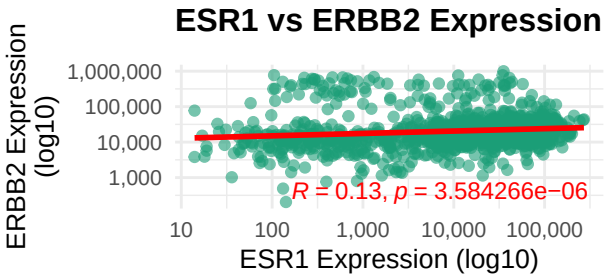
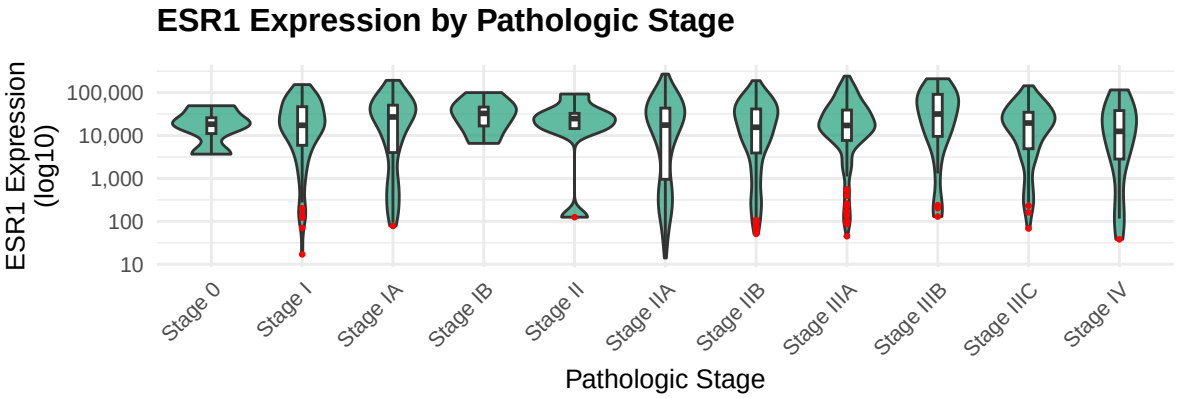


Figure 6




```r
ggsave(
  filename = "SalaamRasheeda-Final Project Check-In Plots.png",
  plot = final_plots,
  width = 12,
  height = 8,
  dpi = 300
)
```

3D. Heatmap Analysis

```r
# Explanation: We have over 1,000 samples, so the heatmap will look busy.
# I've done my best to allow for some readbility by structuring
# the covariate colors to separate into broader pathologic stages.

# Select 10 consecutive genes, starting from ENSG00000091831-- my chosen
# ESR1 gene
which(rownames(tcga_data) == "ENSG00000091831.24")
```

```
## [1] 1944
```

```r
ten_genes <- rownames(tcga_data)[1944:1953]

# Filter covariate data to exclude NAs and X's
filtered_tcga_meta_data <- tcga_meta_data %>%
  filter(
    !is.na(ajcc_pathologic_stage),
    ajcc_pathologic_stage != "Stage X"
  )

# In counts data, replace period delimiter with hyphens
count_barcodes <- gsub("\\.", "-", colnames(tcga_data))

# Grab matching rows from filtered data to create subset below
samples_to_keep <- filtered_tcga_meta_data$barcode

# Subset these 10 genes into a data frame
tcga_data_subset <- tcga_data[
  ten_genes,
  count_barcodes %in% samples_to_keep
  ]

# Convert Ensemble IDs to common gene names
# for readability in heatmap
common_names <- c(
  "ENSG00000091831.24" = "ESR1",
  "ENSG00000091844.8" = "RGS17",
  "ENSG00000091879.14" = "ANGPT2",
  "ENSG00000091947.10" = "TMEM101",
  "ENSG00000091972.18" = "CD200",
  "ENSG00000091986.16" = "CCDC80",
  "ENSG00000092009.10" = "CMA1",
  "ENSG00000092010.15" = "PSME1",
  "ENSG00000092020.10" = "PPP2R3C",
  "ENSG00000092036.18" = "HAUS4"
)
rownames(tcga_data_subset) <- common_names[rownames(tcga_data_subset)]

# Get log of expression values for better readability for Heatmap() below
log_counts <- log10(tcga_data_subset + 1)

# Create heatmap annotations
ha_column <- HeatmapAnnotation(
  Pathologic_Stage = filtered_tcga_meta_data$ajcc_pathologic_stage,
  col = list(
```

```
    # Create a color palette that helps to visualize the broader stages,
    # since we have so many samples
    Pathologic_Stage = c(
      "Stage 0" = "#000000",

      "Stage I" = "#66C2A5",
      "Stage IA" = "#41AE76",
      "Stage IB" = "#238B45",

      "Stage II" = "#8DA0CB",
      "Stage IIA" = "#4F81BD",
      "Stage IIB" = "#225EA8",

      "Stage IIIA"= "#FC8D62",
      "Stage IIIB"= "#F16913",
      "Stage IIIC"= "#D94801",

      "Stage IV" = "#E7298A"
      )
    )
  )

# Save heatmap to PNG with adequate dimensions
png(
"SalaamRasheeda-Final Project Check-in Heatmap.png",
width = 16000,
height = 8000,
res = 1200
)

# Generate heatmap
Heatmap(
  as.matrix(log_counts), # Because Heatmap() expects data in a matrix
  name = "Counts",
  top_annotation = ha_column,
  show_row_names = TRUE,
  row_names_gp = gpar(
    fontsize = 12,
    fontface = "bold"), # I want the common gene names emboldened
  show_column_names = FALSE, # We have 1,000+ samples, so showing the
  # names would be very messy
  cluster_columns = TRUE,
  cluster_rows = TRUE,
  column_title = "Samples",
  row_title = "Genes",
  heatmap_legend_param = list(title = "Count")
)

dev.off()
```

```
## pdf
##   2
```

FINAL SUBMISSION ADDITION: UMAP Plot (ie, new plot type)

```r
# Explanation: For the final project, I want to generate a UMAP to see
# whether samples with similar ESR1 expression also show similar
# overall gene-expression patterns.

# Identify samples that exist in the filtered metadata
matched_samples <- count_barcodes %in%
filtered_tcga_meta_data$barcode

# Confirm that metadata and expression samples are in the same order
all(
  count_barcodes[matched_samples] ==
    filtered_tcga_meta_data$barcode
)
```

```
## [1] TRUE
```

```r
# Keep only samples with valid stage information
tcga_data_filtered <- tcga_data[, matched_samples]

# Log-transform expression values
tcga_data_log <- log10(tcga_data_filtered + 1)

# Calculate variance for each gene
gene_var <- apply(tcga_data_log, 1, var)

# Keep the 2,000 most variable genes
top_genes <- names(
  sort(gene_var, decreasing = TRUE)[1:2000]
)

# Subset data to the most variable genes
tcga_top <- tcga_data_log[top_genes, ]

# Transpose so rows = samples and columns = genes
expr_matrix <- t(as.matrix(tcga_top))

# Run PCA
pca_res <- prcomp(
  expr_matrix,
  center = TRUE,
  scale. = TRUE
)

# Set seed for reproducibility since UMAP
# coordinates can change at each run
set.seed(131)

# Run UMAP
umap_res <- umap(
  pca_res$x[, 1:50],
  n_neighbors = 15,
  min_dist = 0.3
)
```

```r
# Create data frame containing UMAP coordinates
umap_df <- data.frame(
  UMAP1 = umap_res[, 1],
  UMAP2 = umap_res[, 2]
)

# Add ESR1 expression values for the samples included in the UMAP
# Log transform, as with heatmap
umap_df$ESR1 <- log10(esr1_numeric[matched_samples] + 1)

# Create ESR1 UMAP
esr1_umap <- ggplot(
  umap_df,
  aes(UMAP1, UMAP2, color = ESR1)
  ) +
  geom_point(size = 3) +
  scale_color_viridis_c() +
  theme_classic() +
  labs(
    title = "UMAP Colored by ESR1 Expression",
    x = "UMAP 1",
    y = "UMAP 2",
    color = "ESR1 Expression\n(log10-transformed)"
    ) +
  theme(plot.title = element_text(face = "bold",
                                  hjust = 0.5))

# Save plot
ggsave(
  filename = "SalaamRasheeda-FinalProject-ESR1_UMAP.png",
  plot = esr1_umap,
  width = 10,
  height = 7,
  dpi = 300
  )
```

FINAL SUBMISSION ADDITION: Summary statistics (ajcc_pathologic_stage and age_at_diagnosis)

```r
# Explanation: For the final project, I also want to generate
# summary statistics for my pahtologic stage covariate, and
# for age at diagnosis

# Pathologic stage is a categorical variable
# so we can get counts and percentages for each stage, instead...
path_stage_summary_stats <- filtered_tcga_meta_data %>%
  count(ajcc_pathologic_stage) %>%
  mutate(
    Percent = round(100 * n / sum(n), 1)
  )

path_stage_summary_stats
```

```
##    ajcc_pathologic_stage   n Percent
## 1                Stage 0   4     0.4
## 2                Stage I  98     8.8
```

```
## 3               Stage IA  93     8.4
## 4               Stage IB   6     0.5
## 5               Stage II   7     0.6
## 6              Stage IIA 374    33.6
## 7              Stage IIB 267    24.0
## 8             Stage IIIA 155    13.9
## 9             Stage IIIB  27     2.4
## 10            Stage IIIC  62     5.6
## 11              Stage IV  20     1.8
```

```r
# Aside from filtering for NAs below, check for any unexpected age
# values that we'd want to filter out
unique(tcga_meta_data$age_at_diagnosis)
```

```
##    [1] 18475 27865 25477 22536 15774 19989 17523 18733 30407 15969 20678 17702
##   [13] 20332 13307 20764 15432 22958    NA 26723 18282 18296 21628 17652 29751
##   [25] 24999 22849 26221 17080 16955 24439 17474 14883 14264 25830 26883 28495
##   [37] 19478 20820 14316 23230 31805 24262 14122 20570 30783 25186 23521 18237
##   [49] 21926 28715 21793 24053 16174 22942 20717 25029 23058 25119 25010 17640
##   [61] 17070 28609 23318 21465 27163 25770 23928 14532 27121 28063 23150 23660
##   [73] 24923 25350 20152 16884 21394 24232 23008 26052 19706 25259 24877 10898
##   [85] 23576 20705 18985 21910 24660 22788 23404 15052 26121 15705 21608 24891
##   [97] 25242 14969 23753 18482 22462 22422 29146 20065 27934 25966 16817 23000
##  [109] 20924 20497 21646 22411 25475 13483 22707 23507 17468 18524 20381 20719
##  [121] 32478 29734 23485 24647 19123 18492 28664 17763 25937 22629 20468 23653
##  [133] 12142 24219 21197 25643 24850 30125 26186 22199 26484 20694 15812 21623
##  [145] 27917 27400 29444 10832 23500 24779 27964 25190 12528 30958 19152 17033
##  [157] 24319 17224 20105 29474 17807 17689 23922 16504 22905 16736 29014 30590
##  [169] 26684 24577 28411 15602 20471 18177 17727 32842 23675 21949 24104 20930
##  [181] 20425 15780 14564 28247 19298 19192 24279 24719 29006 17817 28706 21428
##  [193] 18852 27421 22133 27837 16615 23650 29144 18506 24541 17699 19731 21837
##  [205] 19607 17921 24519 18025 17440 15837 23927 16101 16508 20077 21228 21294
##  [217] 17539 23883 21201 27729 25097 23055 18594 17710 17238 18659 17363 10812
##  [229] 26466 18839 21479 32462 16702 19781 29345 32208 25345 14503 21865 18345
##  [241] 17332 21622 30742 11204 24788 18046 18109 21731 25944 25957 29330 19758
##  [253] 22539 26931 22710 23560 25294 21122 16511 18270 23224 28444 20940 26896
##  [265] 30085 17022 15216 17580 21426 27063 19131 25959 29199 23741 28981 19516
##  [277] 21762 15540 27455 26278 18900 23108 17045 18354 32750 12840 16689 16978
##  [289] 27489 21391 22643 24353 18152 22179 18384 32872 27811 22115 15524 27731
##  [301] 24155 21534 22524 12443 18538 20574 14250 22896 17490 17806 15324 32295
##  [313] 22703 21575 18433 23358 24417 19031 10564 26845 26535 22751 23686 16719
##  [325] 22344 28855 30196 26724 23294 21641 23225 23103 24791 12715 14854 23337
##  [337] 20715 18084 18519 24831 27288 19114 20776 11650 14595 16742 25974 11929
##  [349] 17981 28724 15670 19643 25355 26021 23215 16603 14784 18558 22809 26999
##  [361] 13556 32061 23530 32871 19389 19266 25598 25811 23543 17996 20173 24731
##  [373] 22779 15942 18362 19074 14920 18691 10616 29966 17521 24235 22355 25000
##  [385] 15050 22416 19975 25158 29820 26269 23057 13138 23116 22928 28625 21463
##  [397] 24411 12965 23065 28923 13445 22052 21857 22900 25142 19900 17779 23423
##  [409] 16377 19805 21369 19027 18628 18507 22995 24242 13208 24315 21389 21488
##  [421] 20539 23126 20374 26514 11379 23011 31404 25666 25110 16697 20075 14119
##  [433] 23284 15744 20049 26993 25747 23639 13805 24181 30859 23164 26550 16757
##  [445] 21348 19010 25499 16882 18611 26399 23327 13671 26007 21397 18401 12668
##  [457] 25362 27047 28304 27557 22340 14174 21856 26299 21464 22842 20482 19880
##  [469] 27521 23371 22071 14374 22598 15212 22810 22397 16829 20028 17426 26869
##  [481] 25289 28403 14494 24125 16195 22380 23708 24288 14778 18850 22023 29256
```

```
##  [493] 16403 12671 17660 16948 29370 25567 24735 28338 27341 19729 15078 29358
##  [505] 23021 17420 22780  9840 11199 22936 29104 14090 25370 20123 15055 24826
##  [517] 32404 28596 20606 17682 25408 18140 22797 20861 25263 22708 12006 20488
##  [529] 26239 22019 18936 22976 19216 15122 24940 17300 27977 19966 18060 26998
##  [541] 17141 26862 17135 23031 24813 25833 18741 20239 29211 24380 22878 19614
##  [553] 25558 24442 17465 19679 12729 15504 16971 18014 13836 21230 18038 21309
##  [565] 30339 26141 23855 28177 20766 23380 22222 15996 13985 29115 21895 26248
##  [577] 23870 20483 19464 19846 20127 15369 19314 16289 15007 18830 22656 31223
##  [589] 24139 19415 24493 21344 25136 18756 28022 16310 22094 27705 14868 16787
##  [601] 18771 18942  9873 17157 23246 14858 23347 13340 17456 27605 25659 23366
##  [613] 27287 23314 25677 28375 22279 17588 19723 20092 16688 17275 17815 19746
##  [625] 19085 21562 18501 18151 24803 18053 20585 19002 22848 21047 21831 23845
##  [637] 18695 18748 23742 24054 24434 16649 20053 21933 20712 29858 22317 23072
##  [649] 13510 20804 21755 14816 22284 25066 18500 24327 20691 21520 21501 22714
##  [661] 17372 25515 16627 15289 17597 25305 12755 19985 24046 15283 29251 22844
##  [673] 16163 21922 28699 22587 22889 29285 15469 22961 12642 16750 20102 18346
##  [685] 16623 18851 16858 18371 20863 18535 19030 19701 19432 26042 18788 29555
##  [697] 19591 19596 20072 23300 15284 26642 15085 20629 24017 24544 19895 16974
##  [709] 14653 23278 21038 21288 25771 24266 20795 17443 22341 13409 17438 24692
##  [721] 21056 14755 10869 16213 17871 23926 23006 18984 30830 21064 23461 23570
##  [733] 16533 22805 26224 20346 20210 18002 16961 25223 19073 25674 18999 23992
##  [745] 23878 25230 26627 24746 29907 20956 20048 14723 23980 12841 22872 21198
##  [757] 21327 14976 23387 17039 13743 19489 17969 19849 19533 21918 22772 22157
##  [769] 17930 29463 17708 22672 22583 24875 24137 21386 20675 20080 20448 12525
##  [781] 19767 28178 23397 26588 16022 21183 28013 18514 15590 26734 19822 17715
##  [793] 28416 28003 27280 22332 17759 27627 21160 18875 30025 23628 18508 23229
##  [805] 14626 25658 28974 22325 15158 29113 29280 21555 15980 24929 20774 18870
##  [817] 14134 16887 17257 19563 28644 20664 19237 25825 16370 18171 22096 26823
##  [829] 13458 20504 22515 21002 16983 24322 31351 19160 15533 14669 20118 15286
##  [841] 22883 13680 12968 16911 25089 26693 18721 28331 18161 22510 11120 24075
##  [853] 22077 29335 17637 14383 14906 28660 26988 21343 30912 15820 16634 23851
##  [865] 19802 18520 21898 22833 19250 21514 20900 23266 19117 13004 13243 20380
##  [877] 19340 15027 15977 16642 25867 27160 27241 23684 14853 23041 22858 19627
##  [889] 29331 22966 22962 16658 28198 17729 24710 22798 29728 29529 18021 21338
##  [901] 23827 31932 25282 27967 27313 21773 13817 14115 25693 24083 15507 30950
##  [913] 22544 26817 22625 18434 13039 29406 17352 26281 27191 20713 29131 23667
##  [925] 20722 14638 19629 27942 27106 18927 13642 21750 31173 19794 18561 21602
##  [937] 31657 22441 17152 30528 18334 28084 18584 22449 16363 31748 27389 24230
##  [949] 15593 26426 30984 20625 23972 14175 17529 19778 15166 16824 18684 20227
##  [961] 19309 17172 22997 19028 17774 18117 21024 19479 21640 28233 26026 17026
##  [973] 22683 22448 15302 21014 18621 28876 20052 28940 18041 19570 15625 14894
##  [985] 29916 16458 16109 20479 30101 20401 20347 30489 19814 27392 14272 27143
##  [997] 22686 23395 22922 14731 15989 26363 17624 25714 20352 18806 19169 22238
## [1009] 21613 22520 17879 27684
```

```r
# Convert ages from days to years
years_old <- tcga_meta_data$age_at_diagnosis/365.25

age_summary_stats <- data.frame(
  mean = mean(years_old, na.rm = TRUE),
  median = median(years_old, na.rm = TRUE),
  sd = sd(years_old, na.rm = TRUE),
  min = min(years_old, na.rm = TRUE),
  max = max(years_old, na.rm = TRUE)
```

```
)

# Round the final values for relevance
round(age_summary_stats, 1)
```

```
##   mean median   sd  min max
## 1 58.9   58.8 13.2 26.9  90
```